



PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS
GRADUAÇÃO EM ENGENHARIA DE SOFTWARE

Arquitetura de Software

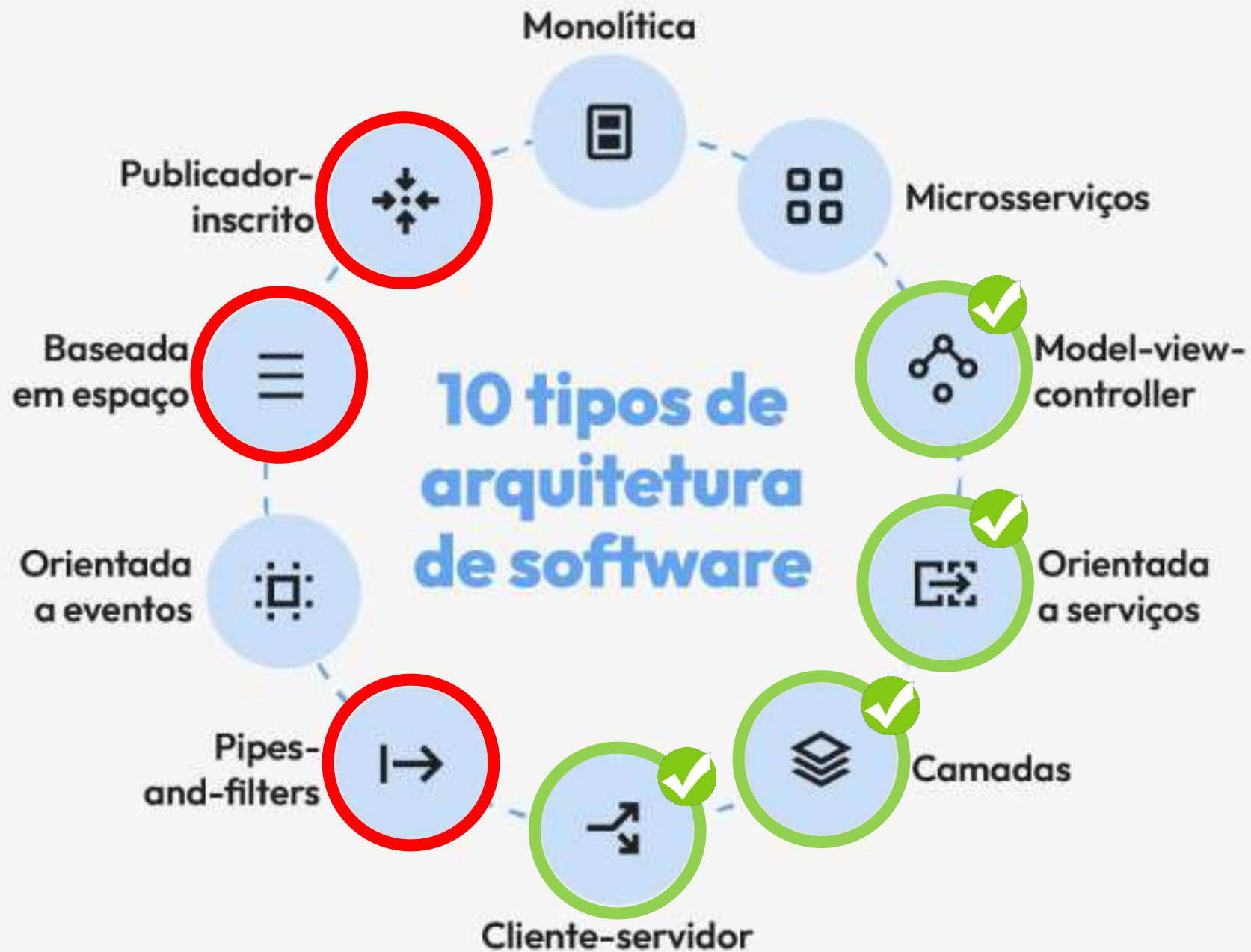
Aula - Estilos Arquiteturais

Pipe-and-Filters, Publish/Subscribe e Space-Based

Prof.: Filipe Nhimi

Agenda

- ❑ Conceitos
- ❑ Exemplos
- ❑ Considerações
- ❑ Conclusão



Pipes-and-Filters





água bruta

coagulantes

tanque de
coagulação
e floculação

tanque de
sedimentação

polímero

controle de corrosão

desinfetante

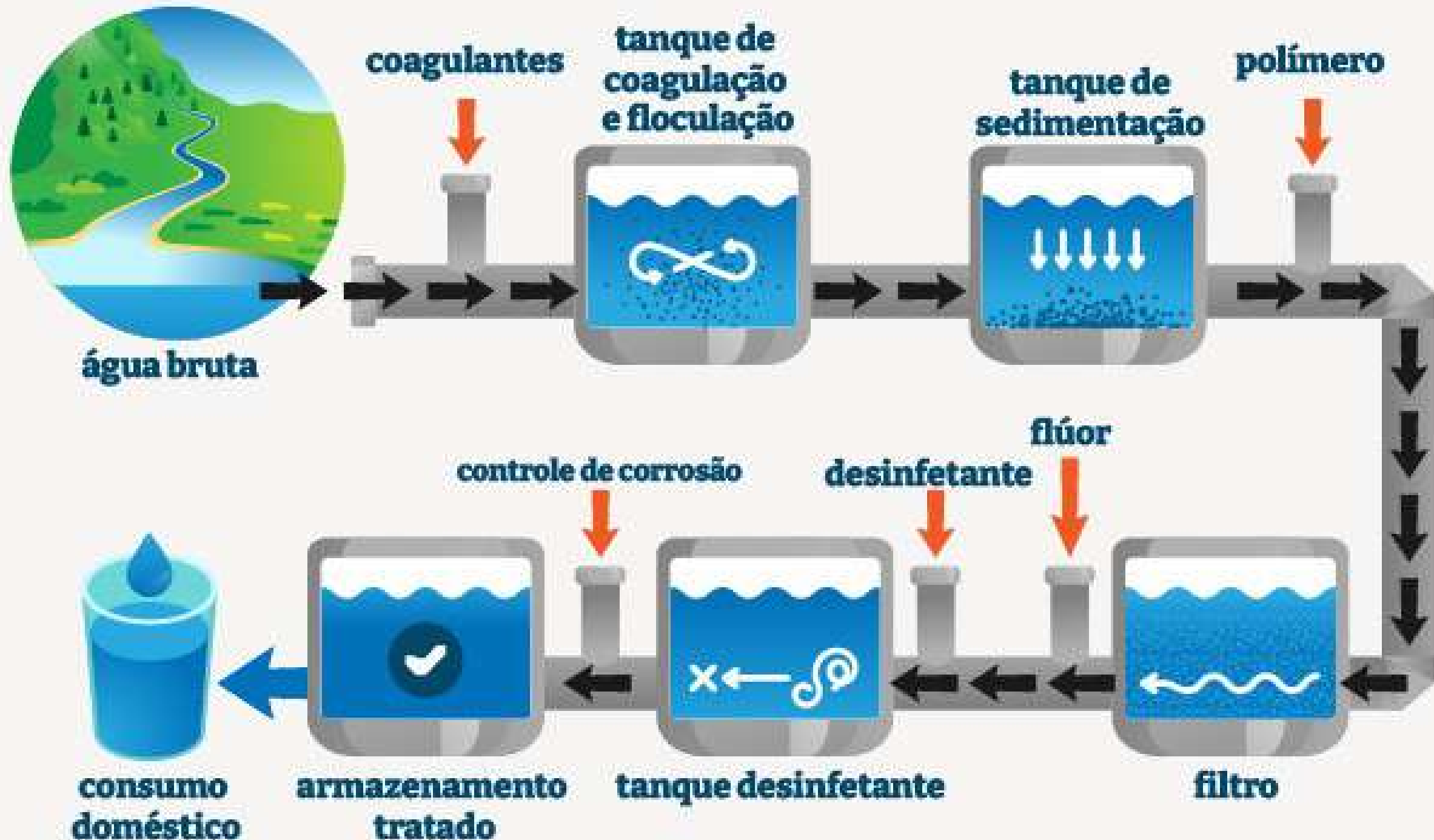
flúor

tanque desinfetante

filtro

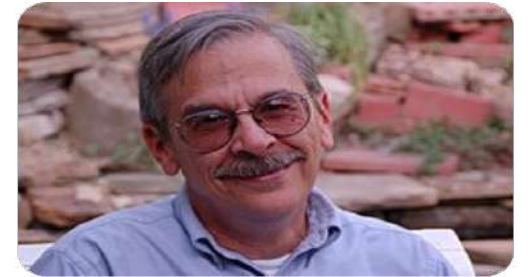
armazenamento
tratado

consumo
doméstico



Estilo Arquitetural *Pipes-and-Filters*

“Segundo Bass, Clements e Kazman, o estilo arquitetural *Pipes-and-Filters* é caracterizado por uma **sequência de componentes independentes (*filters*)** conectados por **canais de comunicação (*pipes*)**, onde cada filtro processa dados de entrada e produz dados de saída para o próximo estágio do fluxo.”



Len Bass



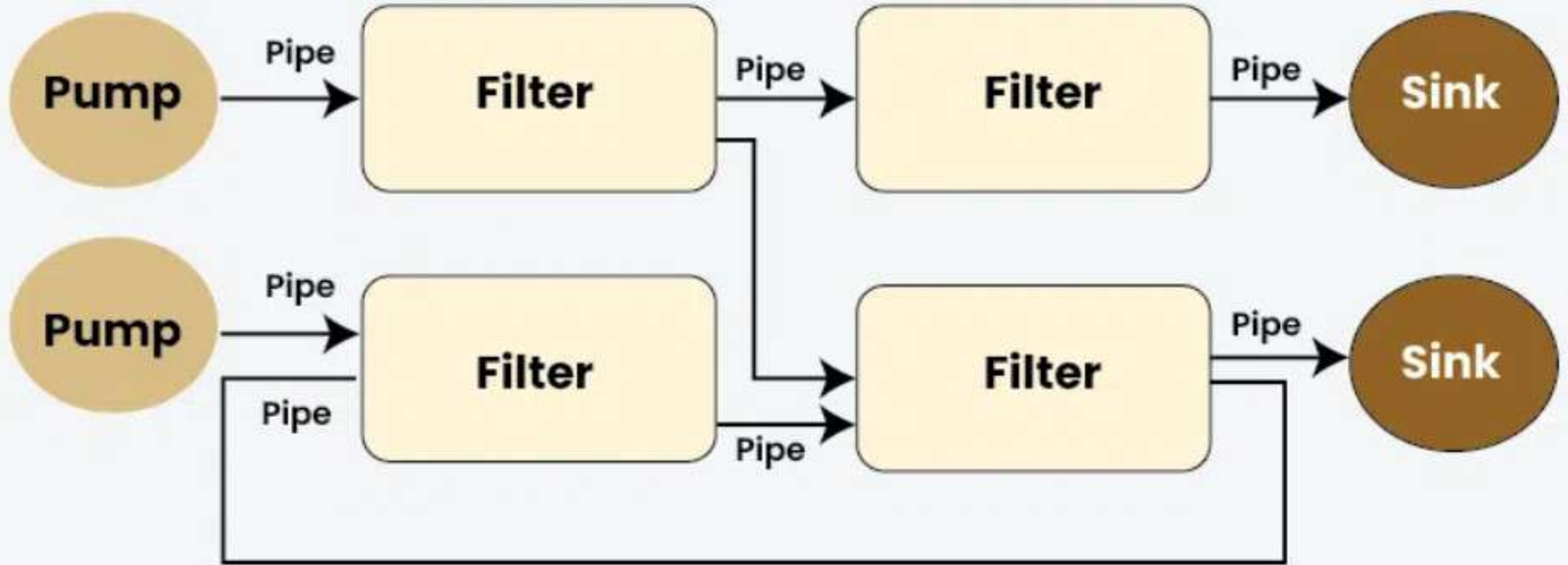
Paul Clements

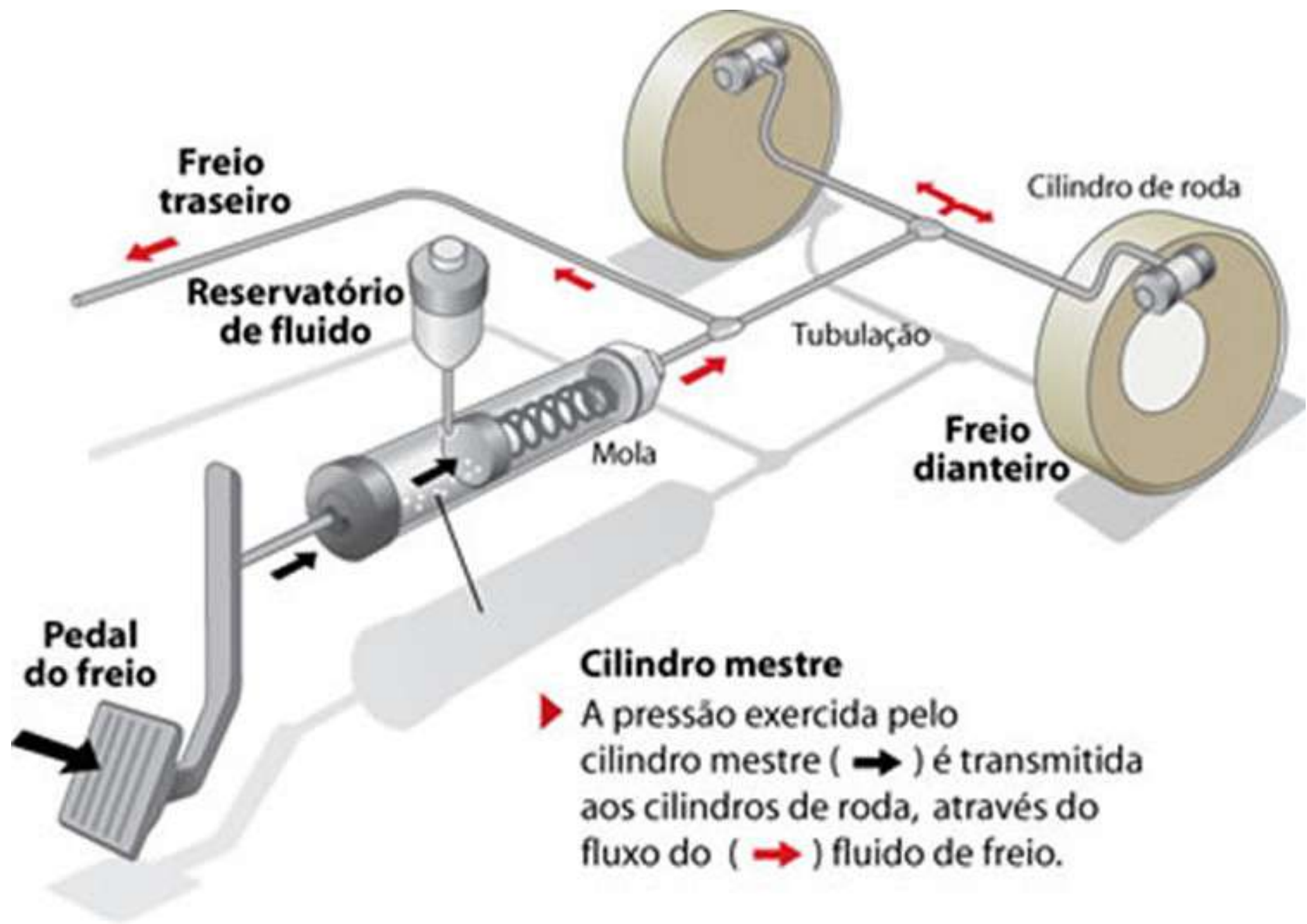


Rick Kazman



Pipe and Filter Architecture - System Design





Componentes

Filtros: Cada filtro **executa uma tarefa específica e independente**, seja **transformar**, **validar** ou **processar** dados antes de encaminhá-los. Na configuração, existem dois níveis de filtros; o primeiro processa os dados da bomba e os passa para o segundo filtro, que continua o processamento antes de repassá-los.

Tubulações: As tubulações servem como **canais para a movimentação de dados entre os filtros**, conectando cada componente em sequência e garantindo uma transferência de dados fluida. Nos diagramas, as tubulações são representadas por setas conectando os componentes.

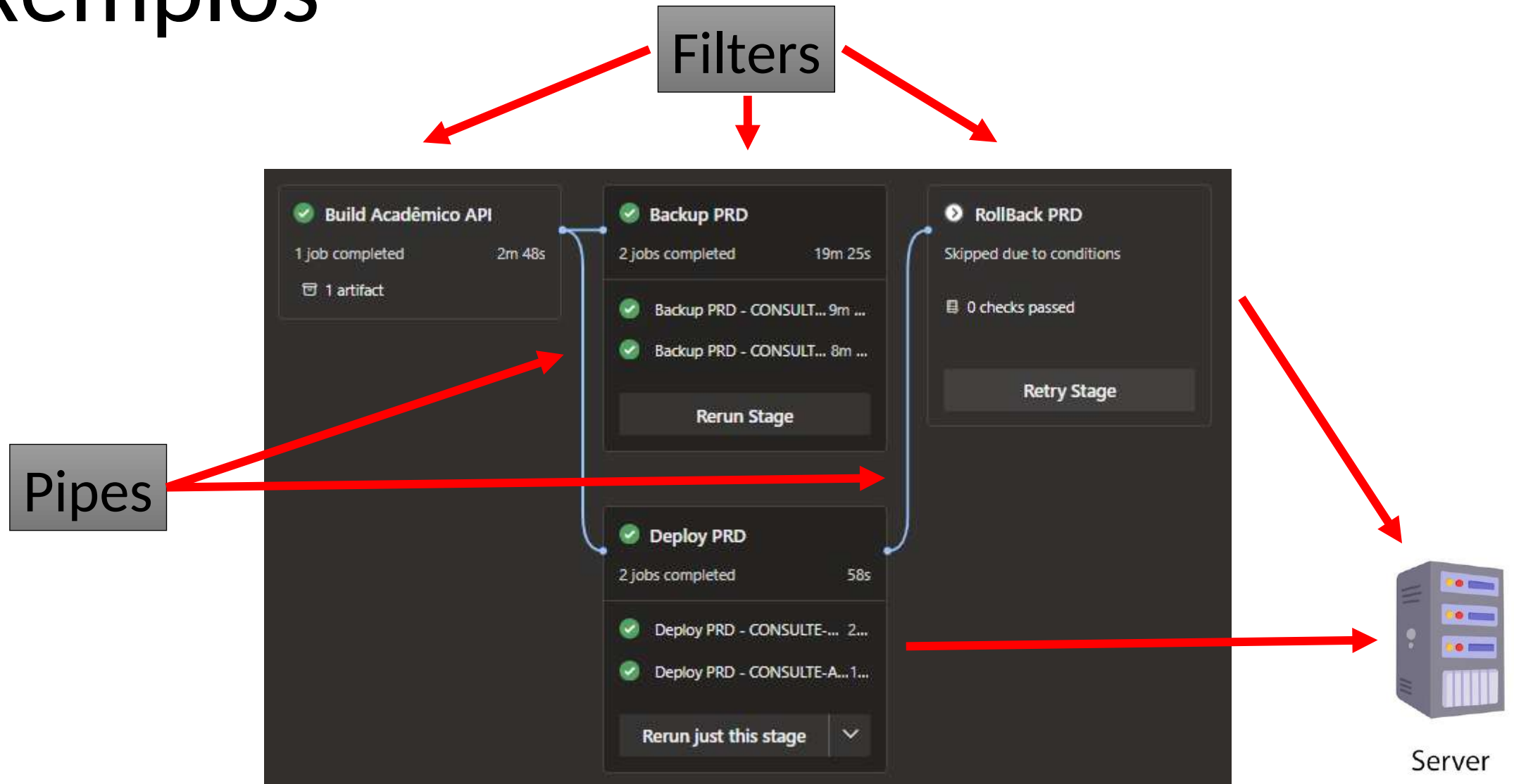
Destinos: Esses são os **pontos finais onde os dados processados são finalmente coletados ou utilizados**. Após passar por todos os filtros, os dados chegam ao destino, completando sua jornada no pipeline.

Componentes

Bombas: Esses componentes **iniciam o processo atuando como fontes de dados**, injetando dados no sistema e iniciando o fluxo através do pipeline.

Processamento Paralelo: A arquitetura também suporta uma estrutura paralela, onde **dois pipelines independentes operam lado a lado**. Cada pipeline começa com sua própria bomba, processa os dados através de uma série de filtros e termina em destinos separados. Isso indica a capacidade de processamento simultâneo de diferentes fluxos de dados sem conflito.

Exemplos



Exemplos

Pump

Azure DevOps consuleinformatica / SAEWEB (NEW) / Repos / Files / saeweb-academico-api-dotnet

SAEWEB (NEW)

- Overview
- Boards
- Repos**
- Files
- Commits
- Pushes
- Branches
- Tags
- Pull requests
- Advanced Security
- Pipelines

saeweb-academico-api-d...

- saeweb-academico-api-dotne
 - API
 - Application
 - Domain
 - Infrastructure
 - Services
 - Test
 - NuGet.Config
 - saeweb-academico-api-dot
 - .gitignore
 - README.md

main

Type to find a file or folder...

Files

Contents History

Graph

Commit

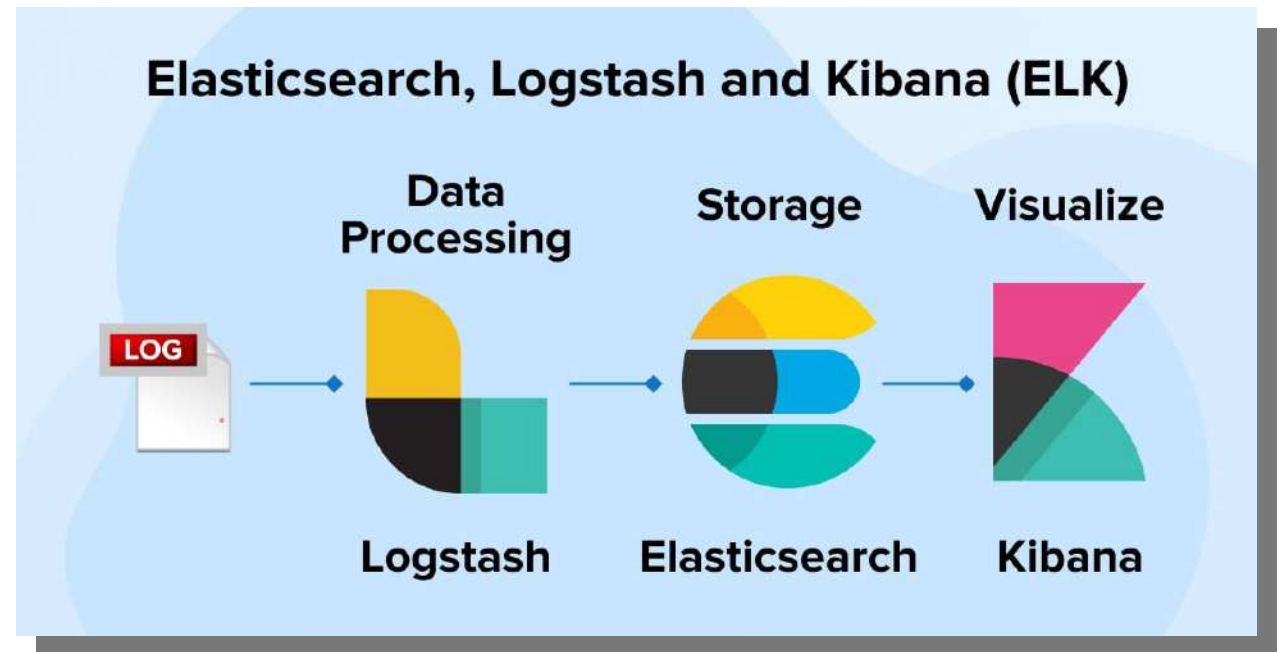
- Merged PR 347: Fix lentidao**
81b096f9 diogo.campos Today at 09:33
- Merged PR 345: Hotfix tela de faltas**
e17d77b0 diogo.campos Yesterday at 17:13
- Merged PR 344: merge**
24c693aa consule.desenvolvimento Yesterday at 14:33
- Merged PR 336: fix: ajustes main**
9d46911e diogo.campos qui. at 12:17
- Merged PR 324: Pesquisa Palavra chave**
a723bda3 Fábio Martins 4 de mai. at 15:01

Exemplos



Centralize, transforme e oculte seus dados

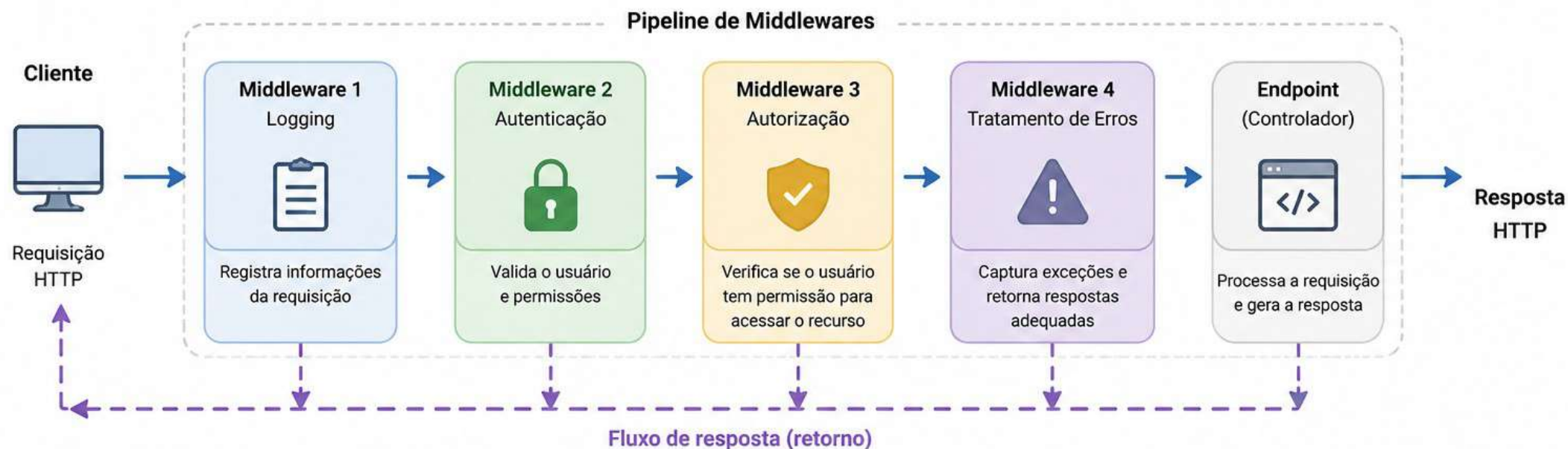
O Logstash é um pipeline open source de processamento de dados do lado do servidor que faz a ingestão de dados de inúmeras fontes, transformá-los e enviá-los para o seu "esconderijo" favorito.



Exemplos

ASP.NET Core – Pipeline de Middlewares

Cada middleware processa a requisição e a encaminha para o próximo componente.



Como funciona

- 1 A requisição HTTP entra no pipeline.
- 2 Cada middleware executa sua lógica.
- 3 O middleware chama o próximo com `await next()`.
- 4 A resposta retorna na ordem inversa, passando pelos middlewares novamente.

Exemplo de configuração (Program.cs)

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

// Ordem dos middlewares importa!
app.UseMiddleware<LoggingMiddleware>();
app.UseMiddleware<AuthenticationMiddleware>();
app.UseMiddleware<AuthorizationMiddleware>();
app.UseMiddleware<ExceptionHandlerMiddleware>();
// ...
app.MapControllers();
app.Run();
```

A requisição passa na ordem de cima para baixo

A resposta retorna na ordem de baixo para cima

Legenda

- Logging
- Autenticação
- Autorização
- Tratamento de Erros
- Endpoint (Aplicação)
- Fluxo da requisição
- Fluxo da resposta

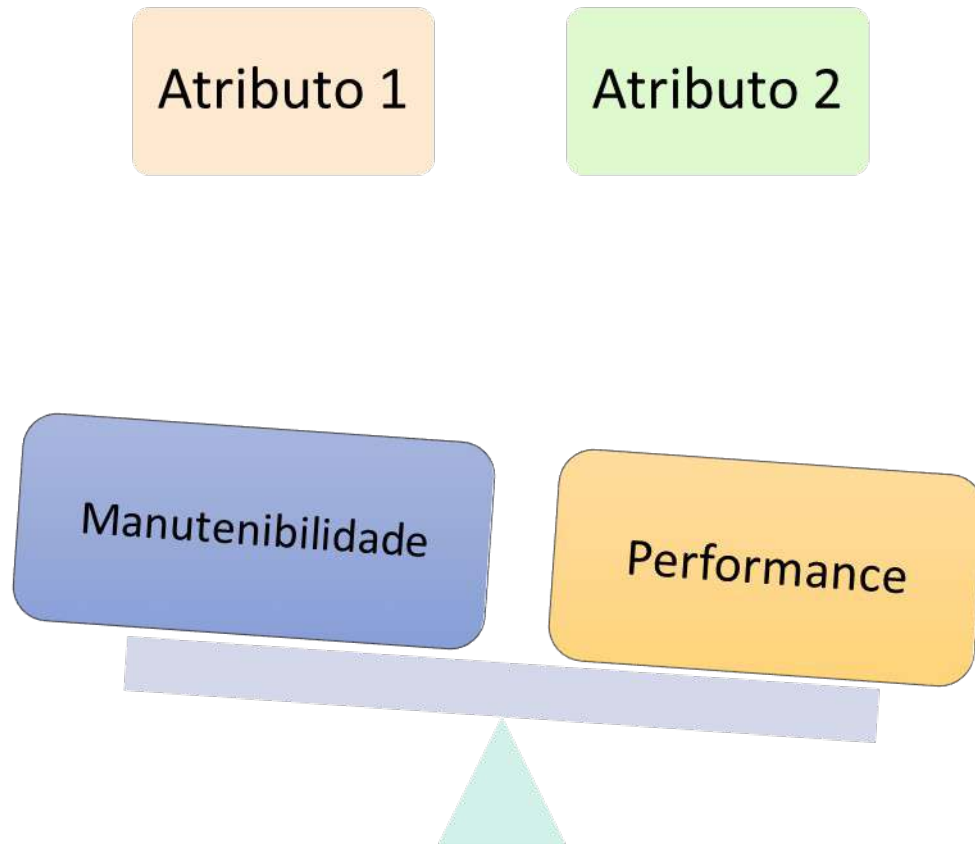
Vantagens (*Pipes-and-Filters*)

- ❑ Alta modularidade
- ❑ Reutilização de filtros
- ❑ Facilidade de manutenção
- ❑ Possibilidade de processamento paralelo
- ❑ Boa composição de pipelines

Desvantagens (*Pipes-and-Filters*)

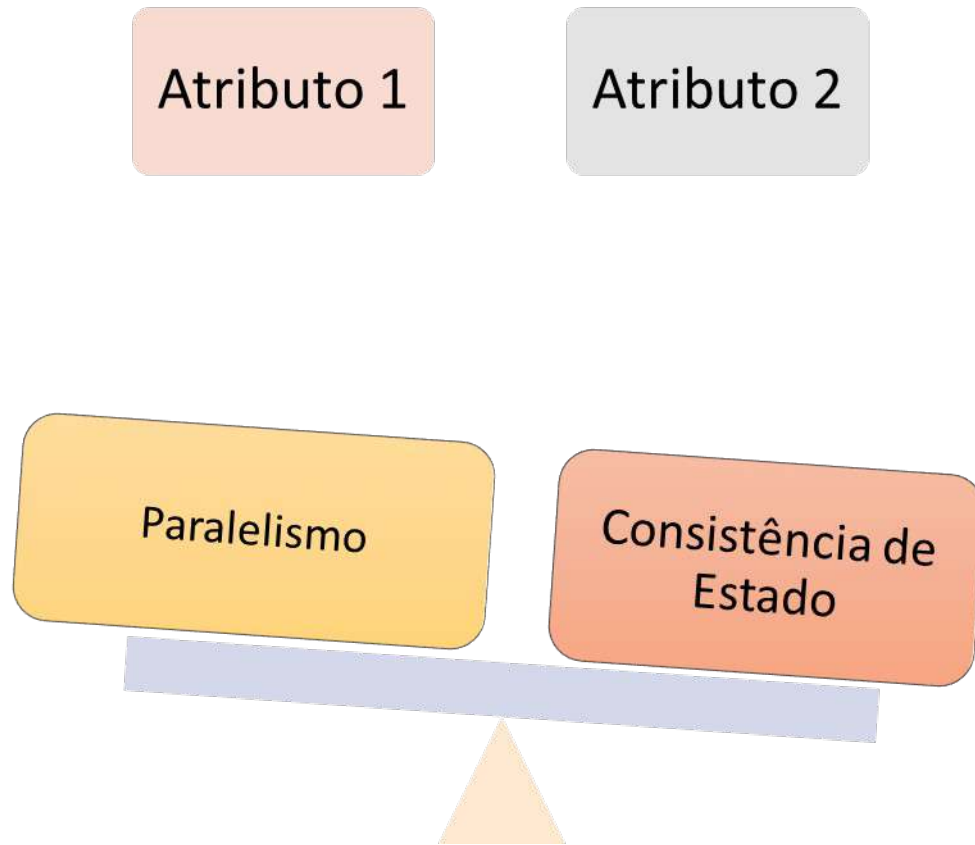
- ❑ Overhead de transformação/transporte de dados
- ❑ Pode aumentar latência
- ❑ Difícil gerenciamento de estado global
- ❑ Nem todo problema se encaixa em fluxo sequencial

Trade-offs do Estilo *Pipes-and-Filters*



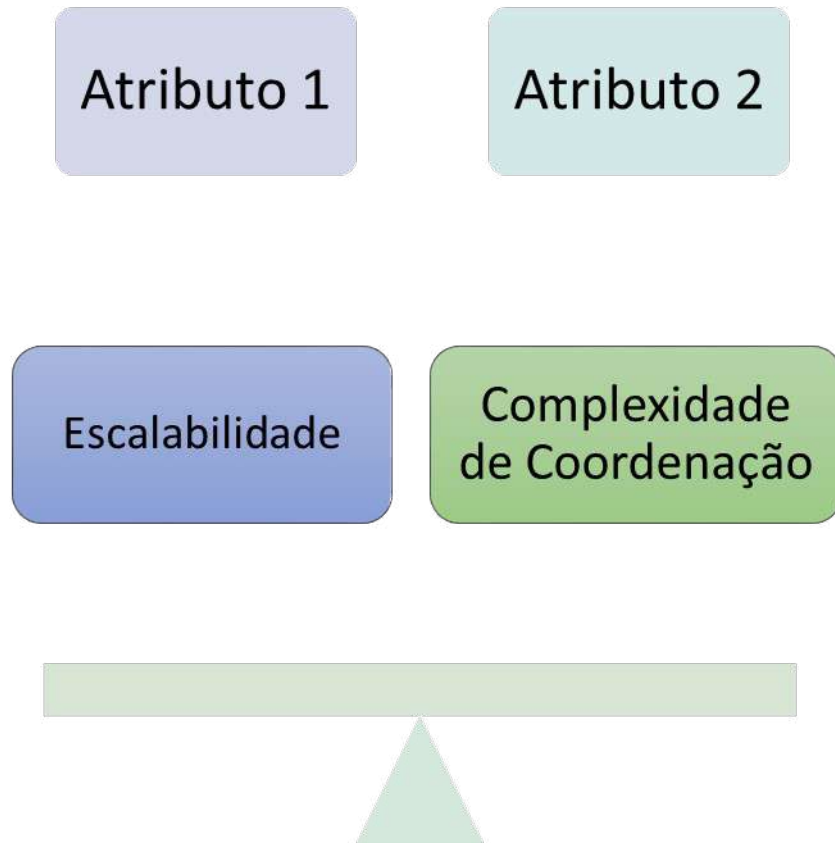
A separação em vários filtros **melhora manutenção e evolução**, porém cada etapa adicional **pode aumentar overhead de processamento**.

Trade-offs do Estilo *Pipes-and-Filters*



O processamento paralelo
melhora desempenho,
porém **dificulta**
gerenciamento de estado
compartilhado

Trade-offs do Estilo *Pipes-and-Filters*



Paralelizar filtros **melhora throughput**, mas **aumenta desafios de sincronização e observabilidade**

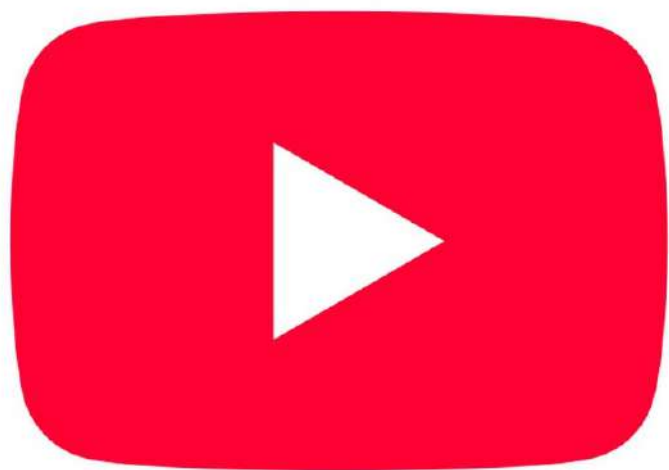
Nem sempre o sistema inteiro adota *Pipes-and-Filters* como estilo arquitetural predominante.

Muitas vezes o padrão aparece apenas em partes específicas da arquitetura, como processamento HTTP, integração, *streaming* ou *pipelines* de build

Conclusão

O estilo arquitetural *Pipes-and-Filters* favorece modularidade, reutilização e composição de processamento através do encadeamento de componentes independentes. É especialmente adequado para fluxos de transformação de dados, pipelines e processamento sequencial, porém pode introduzir overhead, aumento de latência e maior complexidade operacional conforme o pipeline cresce.

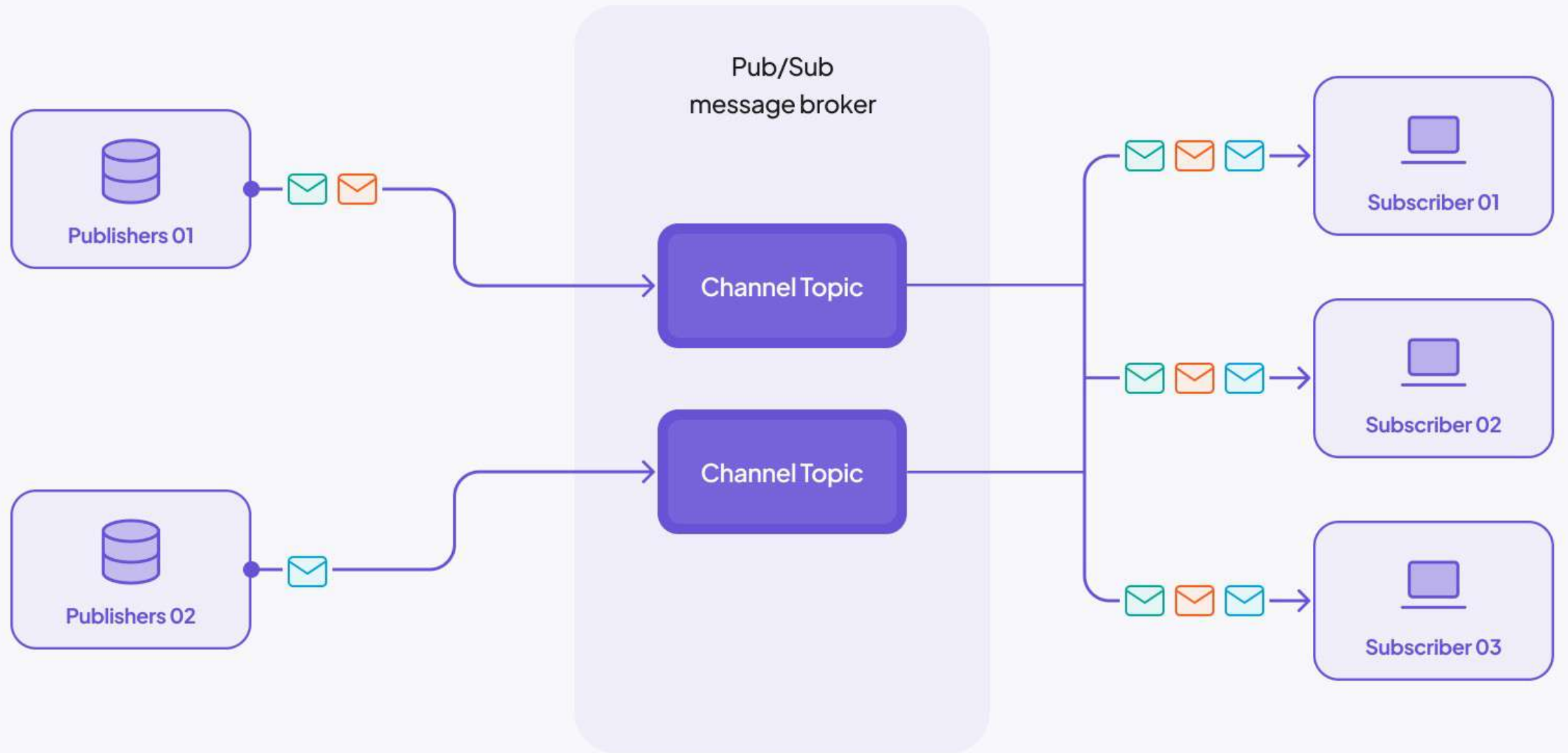
Publish/Subscribe



Pub/Sub

Publish/Subscribe **não é tradicionalmente classificado como um estilo arquitetural clássico**, mas em algumas soluções seu uso pode se tornar tão predominante que passa a definir características arquiteturais importantes do sistema

Ele é um padrão de comunicação
em **arquiteturas orientadas a eventos**



Conclusão (Antecipada... Vamos tratar disso em *EDA – Event Driven Architecture*)

O modelo ***Publish-Subscribe*** promove **baixo acoplamento e comunicação assíncrona** entre produtores e consumidores de eventos, **favorecendo escalabilidade, flexibilidade e integração distribuída**. **Entretanto**, o aumento da assincronicidade pode **dificultar rastreamento de fluxo**, depuração e controle de consistência entre os componentes do sistema.

Space-Based



ARQUITETURA MONOLÍTICA!

SIMPLES, POUCOS RECURSOS, **PORÉM** COM VÁRIOS **PONTOS ÚNICOS DE FALHA**



ARQUITETURA DISTRIBUÍDA

**MAIS COMPLEXA, PORÉM, COM DEDUNDÂNCIA, ALTA
DISPONIBILIDADE E TOLERÂNCIA A FALHAS**

Estilo Arquitetural *Space-Based*

“Segundo Mark Richards e Neal Ford, em *Fundamentos da Arquitetura de Software*, *Space-Based* é um estilo arquitetural projetado para alcançar alta escalabilidade e alta disponibilidade através da remoção de restrições de banco de dados centralizados”



Mark Richards



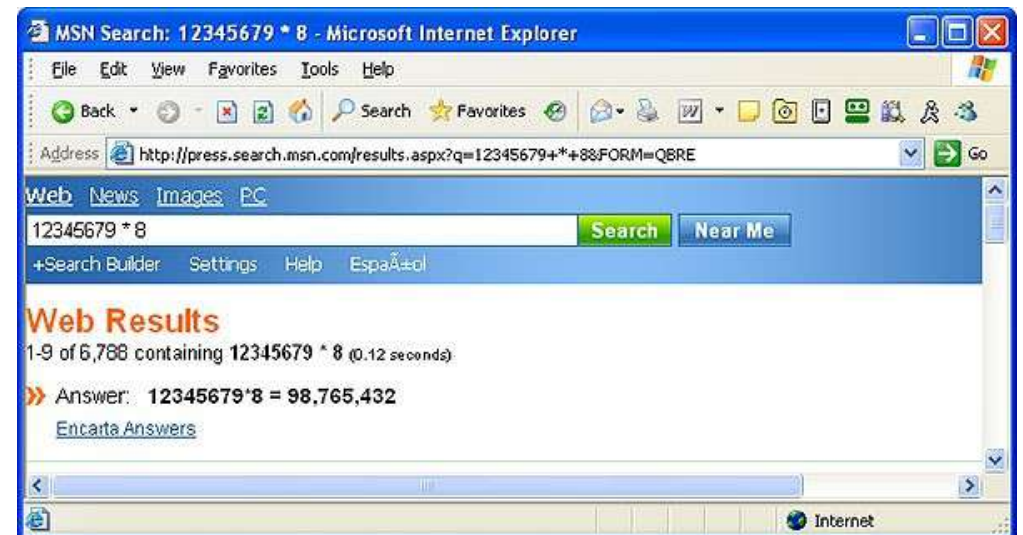
Neal Ford

A arquitetura baseada em espaço (SBA, na sigla em inglês) foi originalmente inventada e desenvolvida na Microsoft em 1997-98. Internamente na Microsoft, era conhecida como plataforma de cache distribuído Youkon (YDC). Os primeiros grandes projetos web baseados nela foram o MSN Live Search (lançado em setembro de 1999)

https://en.wikipedia.org/wiki/Space-based_architecture

1997

Microsoft®



Ideia central

- ☐ Distribuir processamento
- ☐ Distribuir estado
- ☐ Evitar banco central como gargalo
- ☐ Manter dados próximos da aplicação
- ☐ Utilizar memória RAM distribuída

Espalhar as informações e o estado da aplicação entre múltiplos nós do sistema, ao invés de manter tudo centralizado em um único servidor ou banco de dados.

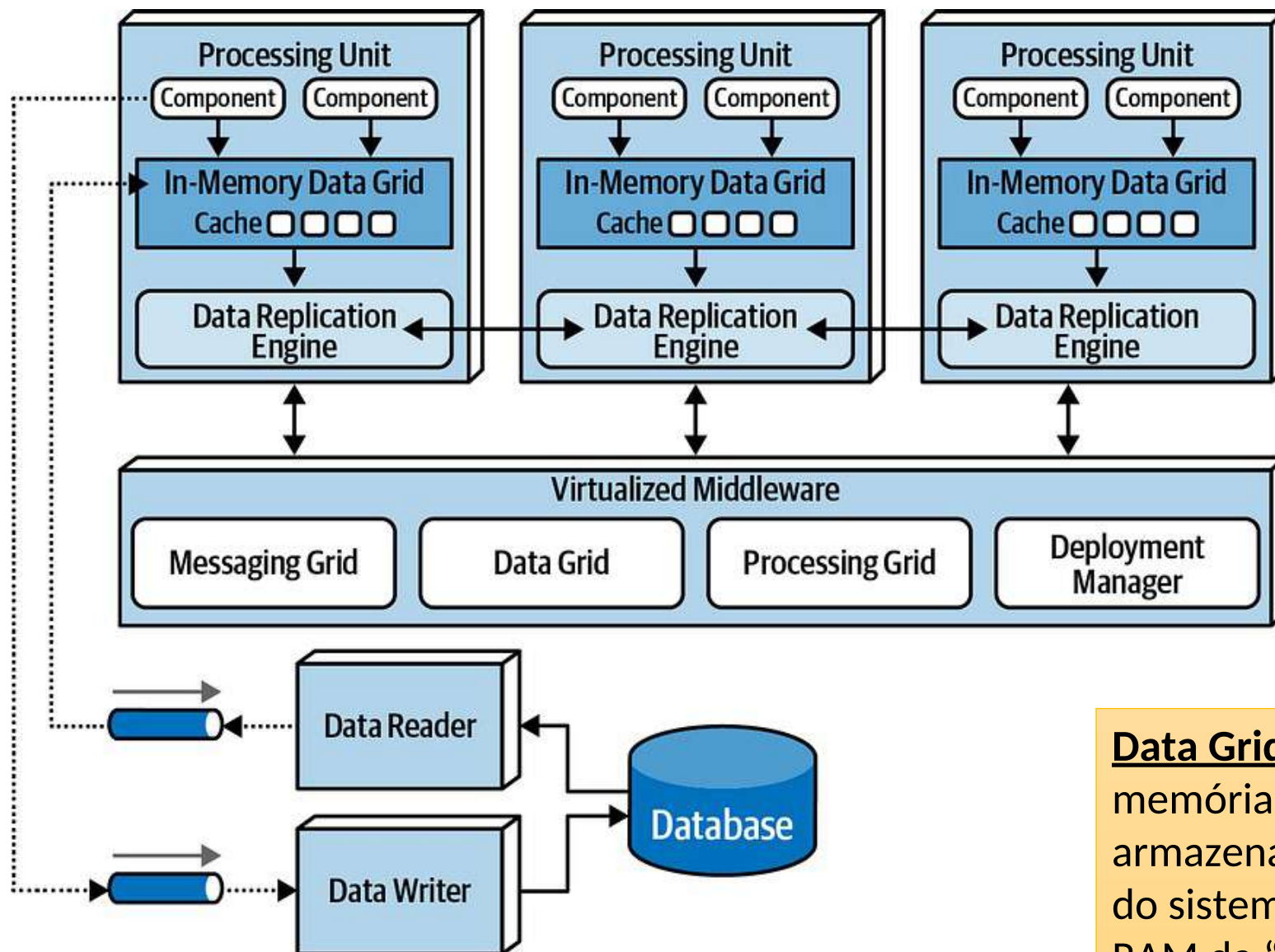
Ex: Sessão de usuários, carrinho de compras, saldo temporário, posição de jogador online, autenticação, conexão ativa, etc.

Ao invés de buscar os dados sempre em um banco centralizado no disco, **o sistema mantém partes dos dados diretamente na memória RAM de vários servidores distribuídos.**

Características do Estilo

- ☐ Altíssima escalabilidade
- ☐ Alta disponibilidade
- ☐ Escalabilidade horizontal
- ☐ Baixa latência
- ☐ Forte uso de memória
- ☐ Processamento distribuído
- ☐ Redução de gargalos centrais

Diagrama SBA



Processing Grid: Distribui a carga computacional entre múltiplas Processing Units para aumentar throughput e escalabilidade.

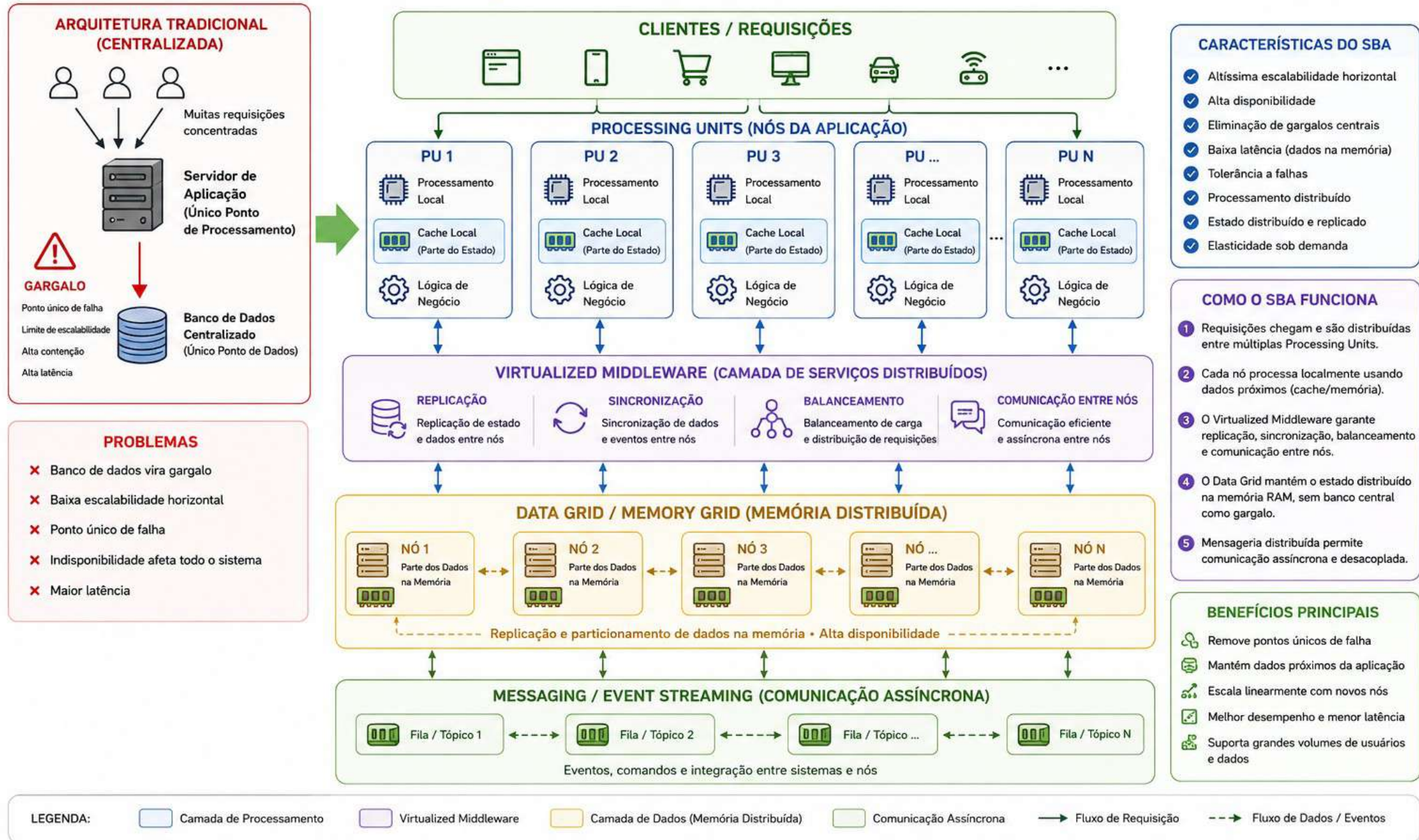
Deployment Manager: Monitora a saúde das instâncias PU para escalar automaticamente

Data Grid: O conjunto distribuído das memórias RAM utilizadas pelas Pus para armazenar e compartilhar estado e dados do sistema. Garante que o dado que está na RAM da “Máquina A” seja copiado para a “Máquina B”.
É ELE QUEM CRIA O CONCEITO DE “ESPAÇO DE DADO!”

Messaging Grid: A infraestrutura responsável pela comunicação assíncrona e troca de mensagens entre as PUs e demais componentes distribuídos.

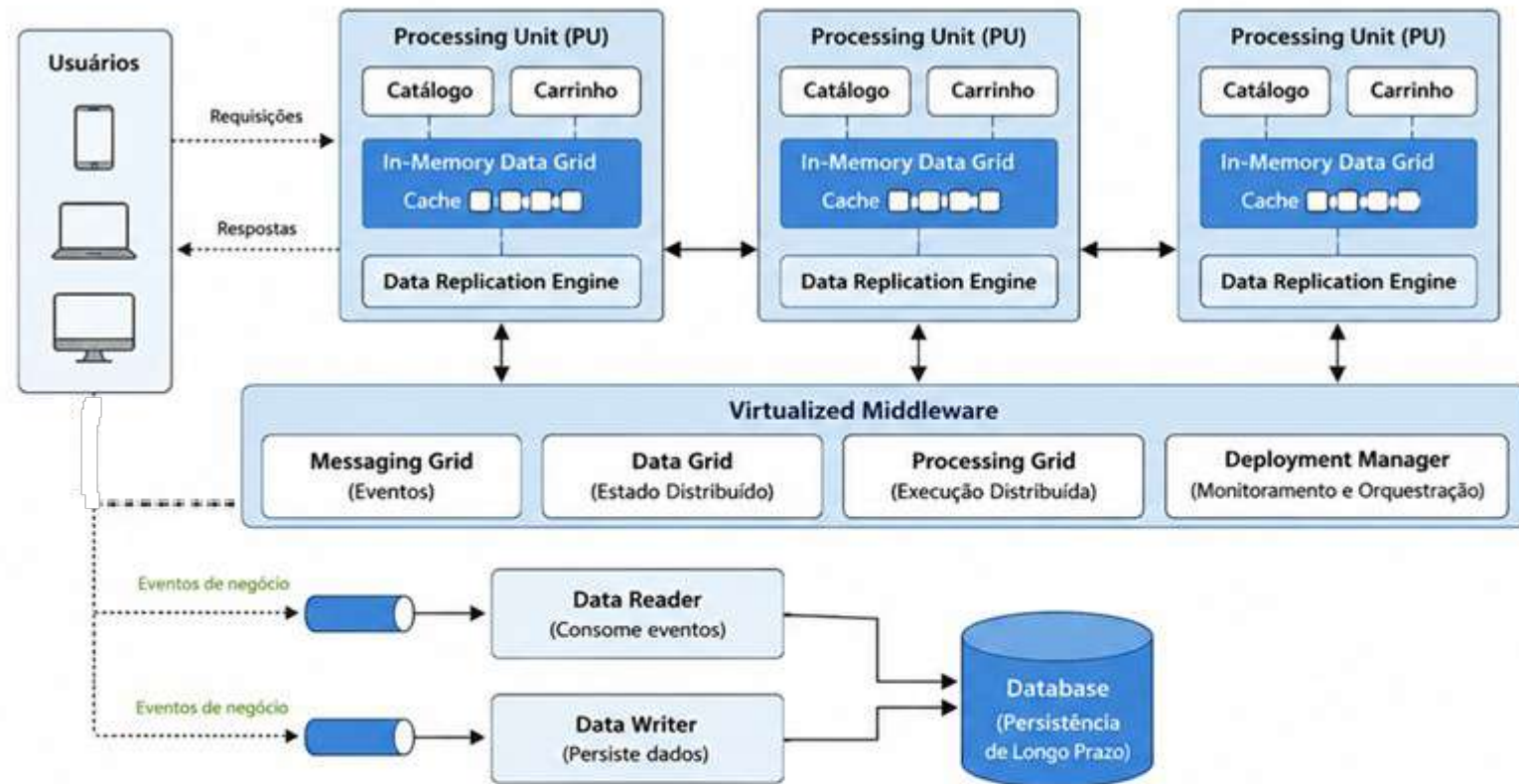
SPACE-BASED ARCHITECTURE (SBA)

Distribuir processamento • Distribuir estado • Evitar banco central como gargalo
Manter dados **próximos** da aplicação • Utilizar **memória RAM distribuída**



Cenário de Exemplo – E-commerce de Alta Escala

Vamos imaginar uma Black Friday de um grande e-commerce



O que cada parte faz?

- Processing Unit (PU)**
Executa a lógica do negócio e mantém parte do estado em memória.
- In-Memory Data Grid**
Armazena em RAM dados como sessões, carrinhos, estoques em tempo real.
- Data Replication Engine**
Replica os dados entre PUs para alta disponibilidade e consistência.
- Virtualized Middleware**
Camada que fornece os serviços distribuídos.
- Messaging Grid:** distribui eventos (e.g. pedido criado, pagamento aprovado).
- Data Grid:** armazena e compartilha estado entre PUs.
- Processing Grid:** distribui tarefas (e.g. cálculo de recomendação).
- Deployment Manager:** monitora, escala e gerencia as PUs.
- Data Reader / Writer**
Lê eventos do sistema e persiste ou sincroniza com o banco.
- Database**
Armazena dados permanentes: pedidos, usuários, histórico, etc.

Fluxo de um Pedido (Exemplo)

- Usuário adiciona produtos ao carrinho e finaliza o pedido.
- A requisição chega a uma Processing Unit (PU).
- A PU grava o estado (ex.: carrinho, pedido) no In-Memory Data Grid.
- A mudança é replicada para outras PUs pelo Data Replication Engine.
- Um evento "Pedido Criado" é publicado no Messaging Grid.
- Outras PUs ou serviços consomem o evento e executam ações em paralelo (ex.: reserva de estoque, cálculo de frete, envio de e-mail) usando o Processing Grid.
- Dados importantes são persistidos no banco via Data Writer.
- Usuário recebe a resposta; o sistema continua escalando conforme a carga.

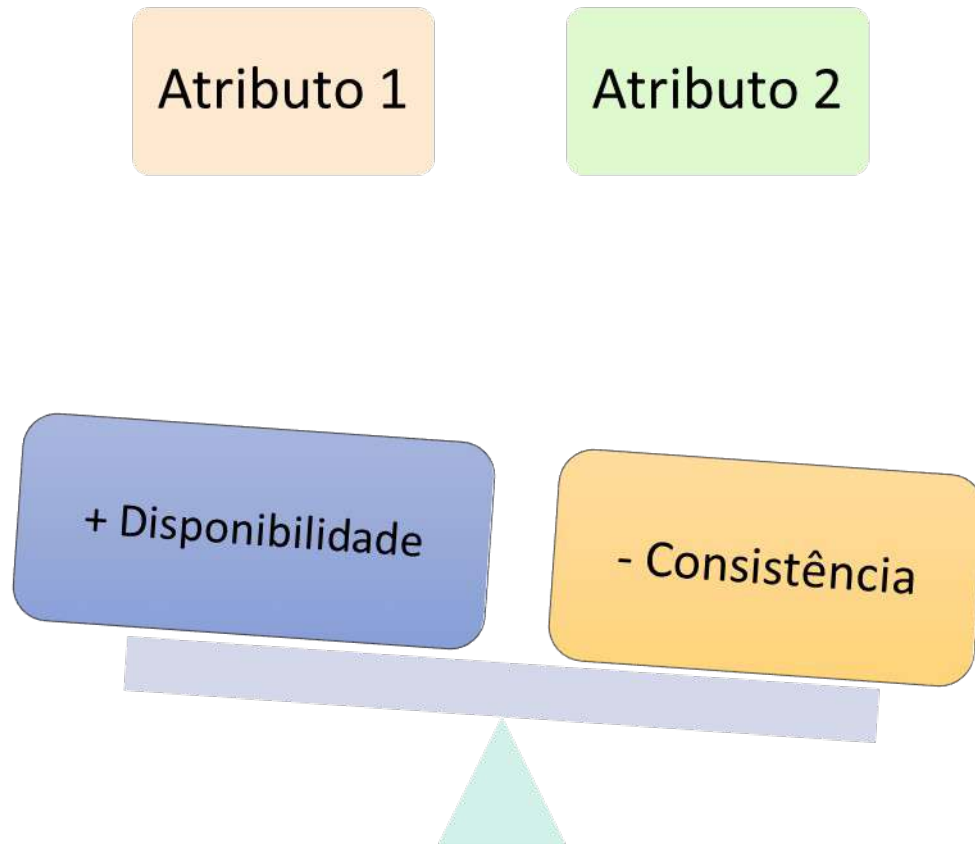
Por que essa arquitetura?

- Escalabilidade:** basta adicionar mais PUs.
- Baixa latência:** dados ficam na memória, próximos do processamento.
- Alta disponibilidade:** dados replicados entre nós.
- Resiliência:** falha de um nó não derruba o sistema.
- Alta performance:** processamento paralelo e eventos assíncronos.

Exemplos de dados no In-Memory Data Grid

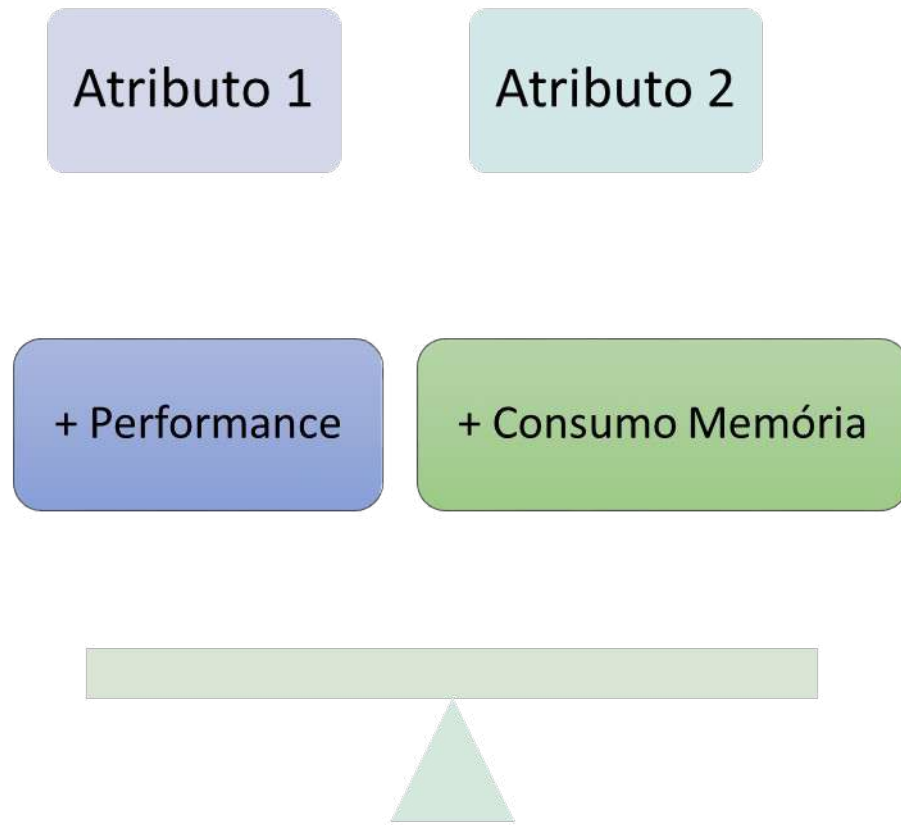
- Sessões de usuários
- Carrinhos de compras
- Estoques em tempo real
- Tokens de autenticação
- Informações de pagamento temporárias
- Rankings e recomendações

Trade-offs do Estilo *Space-Based*



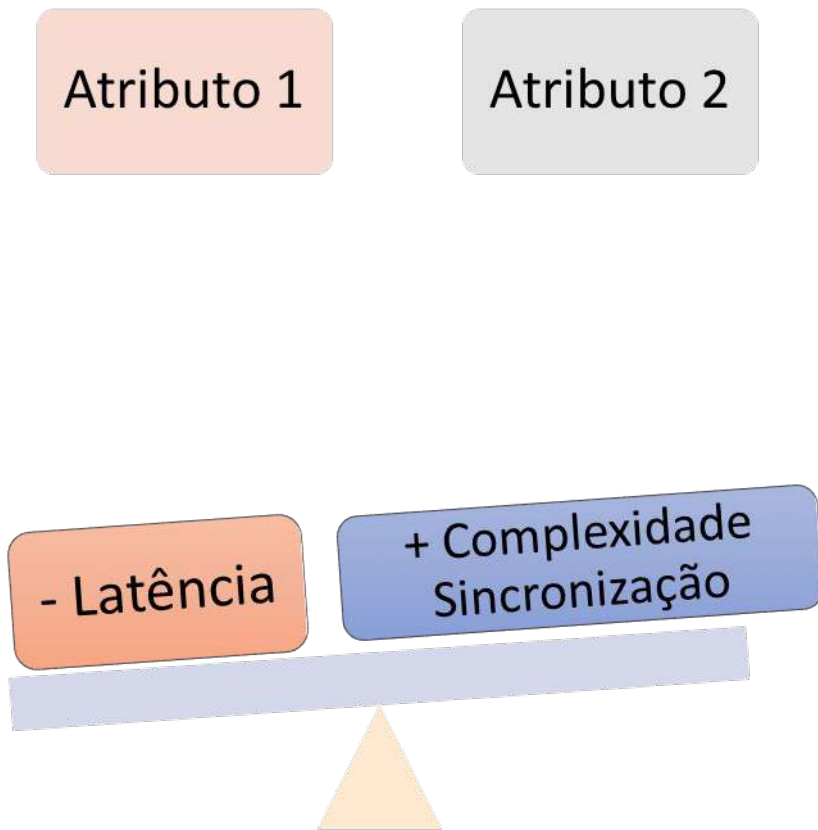
O estilo assume reduzir a consistência forte imediata para **priorizar disponibilidade mesmo quando ocorrer uma partição na rede**, ou seja, AP (Availability + Partition Tolerance)

Trade-offs do Estilo *Space-Based*



Ao priorizar manter dados próximos da aplicação tem-se o **aumento da performance** e **consequentemente maior consumo de memória**

Trade-offs do Estilo *Space-Based*



A latência é reduzida aproximando os dados do processamento, **porém** isso **aumenta o desafio de manter múltiplas cópias sincronizadas.**

SPACE-BASED ARCHITECTURE (SBA)

9. QUANDO USAR | 10. QUANDO NÃO USAR

9. QUANDO USAR

- ✓ Excelente para cenários que exigem **escalabilidade extrema**, **alta disponibilidade** e **baixa latência**.



MARKETPLACES

Grande volume de usuários e transações. Catálogo, busca, carrinho e pedidos distribuídos globalmente.



SISTEMAS FINANCEIROS

Negociação, consultas e operações em alta frequência com tolerância a falhas e baixa latência.



GAMES ONLINE

Ambientes massivos e persistentes, com muitos jogadores simultâneos e estado distribuído.



STREAMING

Entrega de conteúdo para milhões de usuários com baixa latência e alta disponibilidade.



IOT (INTERNET DAS COISAS)

Milhões de dispositivos enviando e recebendo dados continuamente.



SISTEMAS COM PICO EXTREMO

Eventos sazonais ou campanhas que geram picos massivos de acessos e transações.



APLICAÇÕES SAAS MASSIVAS

Sistemas multi-tenant com necessidade de crescimento contínuo e elasticidade.



EM RESUMO

Use SBA quando o objetivo principal for **ESCALAR SEM LIMITES**, garantindo **ALTA DISPONIBILIDADE** e **RESILIÊNCIA**.

10. QUANDO NÃO USAR



Evite SBA quando a complexidade e o custo não se justificam para o contexto do sistema.



SISTEMA PEQUENO

Para sistemas simples, o overhead e a complexidade do SBA não trazem benefícios reais.



BAIXA CONCORRÊNCIA

Quando o número de usuários e requisições é baixo, uma arquitetura mais simples é suficiente.



CONSISTÊNCIA FORTE OBRIGATÓRIA

Se o negócio exige consistência forte em tempo real (ACID estrito), o SBA pode não ser adequado.



EQUIPE PEQUENA

SBA requer conhecimento especializado para operar e manter ambientes distribuídos complexos.



ORÇAMENTO LIMITADO

Infraestrutura distribuída, ferramentas e operação têm custo mais alto.



EM RESUMO

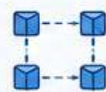
Evite SBA quando o custo, a complexidade e os requisitos de consistência superarem os benefícios de escalabilidade e resiliência.

SBA vs EDA vs MICROSERVIÇOS

Três abordagens complementares que resolvem problemas diferentes e podem coexistir na mesma solução.

1. SPACE-BASED ARCHITECTURE (SBA)

Foco: Escalabilidade extrema, baixa latência e alta disponibilidade



IDEIA CENTRAL

Distribuir processamento, estado e memória para eliminar gargalos centrais.

CARACTERÍSTICAS PRINCIPAIS

- Memória distribuída (Data Grid)
- Replicação e particionamento de estado
- Processing Units (nós) com cache local
- Virtualized Middleware (replicação, sincronização, balanceamento, comunicação)
- Alta tolerância a falhas e disponibilidade
- Evita banco central como gargalo
- Baixa latência (dados próximos da aplicação)



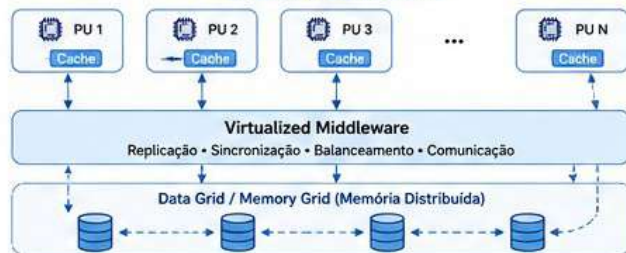
PROBLEMA QUE RESOLVE

"Meu sistema não escala porque tudo depende de um componente central."

PALAVRA-CHAVE

Distribuição de estado e processamento

DIAGRAMA CONCEITUAL



2. EVENT-DRIVEN ARCHITECTURE (EDA)

Foco: Comunicação assíncrona, desacoplamento e reatividade



IDEIA CENTRAL

Componentes publicam eventos e outros componentes consomem esses eventos.

CARACTERÍSTICAS PRINCIPAIS

- Comunicação assíncrona baseada em eventos
- Desacoplamento entre produtores e consumidores
- Uso de brokers/mensageiros (ex.: Kafka, RabbitMQ)
- Reatividade: reage a acontecimentos do sistema
- Melhor resiliência (filas, reprocessamento)
- Fluxo de dados e integração entre serviços



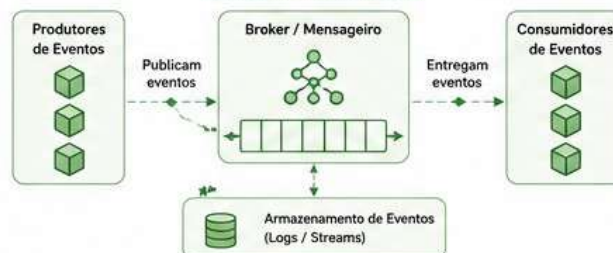
PROBLEMA QUE RESOLVE

"Quero desacoplar componentes e reagir a acontecimentos do sistema."

PALAVRA-CHAVE

Comunicação orientada a eventos

DIAGRAMA CONCEITUAL



3. MICROSERVIÇOS

Foco: Modularização funcional, independência e entrega contínua



IDEIA CENTRAL

Dividir o sistema em pequenos serviços independentes e coesos por domínio.

CARACTERÍSTICAS PRINCIPAIS

- Serviços pequenos e independentes
- Cada serviço com sua responsabilidade (bounded context)
- Deploy e escalabilidade independentes
- Banco de dados por serviço
- Times autônomos por serviço
- Comunicação via APIs (sync ou async)



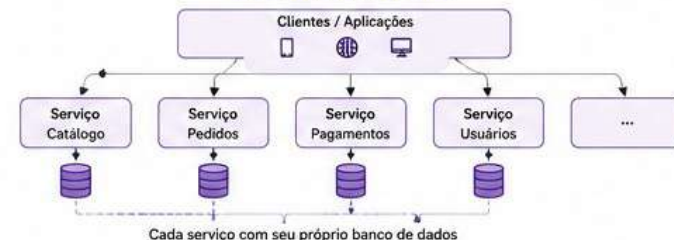
PROBLEMA QUE RESOLVE

"Meu sistema monolítico está difícil de evoluir e manter."

PALAVRA-CHAVE

Separação funcional do domínio

DIAGRAMA CONCEITUAL



ASPECTO	SBA	EDA	MICROSERVIÇOS
O QUE DISTRIBUI	Estado, memória e processamento	Eventos e mensagens	Responsabilidades de negócio (serviços)
PRINCIPAL OBJETIVO	Escalabilidade extrema e baixa latência	Desacoplamento e reatividade	Modularidade, autonomia e evolução independente
COMUNICAÇÃO	Entre nós (middleware distribuído)	Assíncrona (eventos via broker)	Síncrona (APIs) e/ou Assíncrona (eventos)
CONSISTÊNCIA	Geralmente eventual (consistência relaxada)	Eventual (por natureza assíncrona)	Depende da implementação de cada serviço
EXEMPLOS DE USO	Sistemas financeiros de alta frequência, e-commerce massivo, jogos online, IoT, redes sociais, streaming	Integrações, pipelines de dados, notificações, monitoramento, IoT, processos de negócio	Aplicações complexas, domínios grandes, produtos com evolução contínua

ELES PODEM COEXISTIR!

Não são alternativas excluintes.
É comum ver arquiteturas como:

- ✓ Microserviços + EDA
- ✓ SBA + EDA
- ✓ Microserviços + SBA
- ✓ SBA + Microserviços + EDA

★ Cada abordagem resolve um problema diferente.



RESUMO FINAL:

Microserviços organiza responsabilidades de negócio, EDA organiza a comunicação entre componentes e SBA organiza a distribuição de processamento e estado para atingir escalabilidade extrema, alta disponibilidade e baixa latência.

Conclusão

O estilo arquitetural *Space-Based* **busca eliminar gargalos centralizados** através da distribuição de processamento e estado entre múltiplas instâncias da aplicação. É altamente eficiente para sistemas com grande volume de acessos e necessidade de escalabilidade extrema, **porém aumenta significativamente a complexidade de sincronização, observabilidade e gerenciamento da infraestrutura distribuída.**